

Module 6: Functions I

Introduction to C++ Programming — A Beginner's Course

Module Overview

Functions are introduced as the direct answer to a pain point students already feel: they've been copy-pasting similar logic (like averaging a list) across their Module 5 project. Anchor the entire module in 'don't repeat yourself' as a motivating principle, then work through the mechanics (parameters, return types, void, prototypes, pass-by-value) as the toolset for achieving it.

Suggested Duration: 60–75 minutes

Learning Objectives

By the end of this module, students will be able to:

- Explain the DRY (don't repeat yourself) motivation for functions.
- Write a function with correctly typed parameters and a return type.
- Distinguish parameters (in the definition) from arguments (at the call site).
- Write and use void functions appropriately.
- Use function prototypes to allow calling a function before its full definition.
- Explain pass-by-value and predict that a function cannot modify its caller's variable by default.

Prerequisites

- Module 5: recognizing repeated logic across a program (e.g. computing an average multiple times).

Materials & Instructor Prep

- 06-functions-i/examples/03_pass_by_value.cpp preloaded for the pass-by-value surprise demo.

Key Vocabulary

Term	Definition
Function	A named, reusable block of code that performs a specific task.

Term	Definition
Parameter	A named placeholder in a function's definition for a value the caller must supply.
Argument	The actual value supplied at the call site, matched to a parameter.
Return type	The type of value a function hands back to its caller (void if none).
Function prototype	A function's signature only (no body), declared before it's used so the compiler recognizes it ahead of the full definition.
Pass by value	The default C++ argument-passing behavior: the function receives a copy, not the original variable.

Lesson Outline & Instructor Talking Points

1. Motivate with DRY, using their own Module 5 code (10 min)

- If possible, pull up an actual student's Module 5 grade tracker (with permission) or your own reference solution, and highlight two near-identical blocks computing an average.
- State the principle: 'If you ever copy-paste code and change one small thing, that's a signal you need a function instead.'

Instructor note: *This concrete callback to their own recent work lands far better than an abstract definition of DRY.*

2. Anatomy of a function (15 min)

- Build `int add(int a, int b) { return a + b; }` live, labeling each part explicitly: return type, name, parameter list, body, return statement.
- Immediately call it: `int result = add(3, 4);`. Introduce parameter vs. argument terminology here, using this exact call as the concrete example.

3. void functions (10 min)

- Contrast a value-returning function with a void one that only prints. Ask: 'When would you want a function that doesn't hand anything back?' before giving the answer (side-effect-only actions like printing or modifying state directly).

4. Declaration order and prototypes (10 min)

- Show a function called before its definition, without a prototype — let the compiler error happen live.

- Add the prototype line above main and re-run successfully. State the underlying rule plainly: 'declared before used,' with the prototype as one way to satisfy it.

5. Pass by value — the surprise (15–20 min)

- Live-run examples/03_pass_by_value.cpp. Before revealing output, ask the class to predict whether number changes after tryToDouble(number) — most will guess yes.
- Reveal that it's unchanged, and explain: the function received a COPY. Changing the copy never touches the original.
- Frame this explicitly as a safety feature, not a limitation: 'A function can't accidentally corrupt your data unless you deliberately ask it to' — and preview that Module 7 introduces exactly that deliberate mechanism (references).

Instructor note: *This is the module's 'aha' moment, structurally parallel to Module 2's integer-division surprise. Don't rush it.*

Common Student Misconceptions

Misconception	How to Address It
"Changing a parameter inside a function changes the caller's variable."	By default, arguments are copied (pass by value) — demonstrate live, and preview that Module 7 covers the deliberate exception (pass by reference).
"A function's parameter names must match the caller's variable names."	Show a call using differently-named variables/literals to prove parameter names are purely internal to the function.
"void means the function does nothing."	void means it returns no value — it can still print, modify state, or produce visible effects.

Exercises & Module Project

Exercise: 06-functions-i/exercises/exercise1.cpp — isEven, average(vector), printLine functions.
 Project: a Math Toolkit (06-functions-i/project/README.md) with 5 required functions, using the prototype-above/definition-below pattern throughout.

Facilitation guidance: Require students to write every prototype in the Math Toolkit project even though it's technically only strictly necessary for functions called before their definition — the repetition builds the habit before it's optional.

Discussion & Comprehension Check Questions

1. What's the difference between a parameter and an argument?

Guidance: Parameter = the named placeholder in the function's own definition; argument = the actual value passed in at a specific call site.

2. Why doesn't changing a function's parameter affect the variable the caller passed in?

Guidance: By default, C++ passes arguments by value — the function works on a copy.

Assessment Ideas

- Given a function signature only (e.g. `double area(double length, double width)`), have students write a valid call site using their own variable names.

Answer Key Notes

See `06-functions-i/exercises/solutions/exercise1_solution.cpp`.

Wrap-Up & Bridge to Next Module

Bridge forward: "Sometimes a function DOES need to change the caller's variable, or hand back more than one result. Next module shows you the deliberate, on-purpose way to do that — plus recursion, a function that calls itself."

Instructor Tips

- Keep the DRY framing alive throughout the rest of the course — every later module's project should be graded partly on 'did you factor out repeated logic into a function,' not just correctness.